

Smart MCQ Solver: Comparative Architecture & Performance Analysis

1. Introduction & Problem Statement

I built this project to solve a ranking problem, not a plain classification problem. Every question in the Smart MCQ Solver dataset comes with a prompt and five candidate answers, labeled A through E. Only one option is correct, but the competition doesn't just want my best guess — it wants my top three guesses, in order. The evaluation metric is Mean Average Precision at 3, or MAP@3, which gives full credit if I put the right answer first, half credit if it lands second, a third of a point if it lands third, and nothing if it's outside the top three. That scoring rule shaped almost every design decision I made, because a model that's "mostly right but unsure" can still score well here, as long as it keeps the correct answer near the top of its ranked list.

The project rules required three separate models, each built a different way. One had to be built completely from scratch, with no pretrained weights anywhere in the architecture. One had to be a pretrained transformer, fine-tuned on this specific dataset. The third was open-ended — any additional approach I wanted to try, as long as it was structurally distinct from the first two. I also had to run all training and inference locally or on Kaggle's own GPU kernels, with zero calls to any external LLM API. No OpenAI, no Gemini, nothing hosted. Every prediction had to come from code running inside my own notebook.

I ended up building three genuinely different systems: a transformer encoder I wrote by hand in PyTorch, a fine-tuned `bert-base-uncased` model using Hugging Face's multiple-choice head, and a gradient-boosted tree ensemble built on top of engineered features and sentence embeddings. Two of the three beat the 0.73 qualification cutoff, and I logged every training run in Weights & Biases so I could actually compare them side by side instead of just trusting my gut about which one worked better.

What follows is a breakdown of how each model is built, how they performed against each other, what it costs computationally to run each one, and how I'd actually get this thing in front of a user instead of leaving it stuck inside a notebook.

2. Core Model Architectures & Performance

2.1 Model 1 — The Scratch Transformer

I wrote Model 1 without touching Hugging Face's `transformers` library for the architecture itself. Every layer — the embedding table, the positional encoding, the attention blocks — is plain PyTorch, initialized from random weights and trained only on this dataset. The vocabulary is built with Python's `Counter`, capped at the 10,000 most frequent words, with reserved slots for padding and unknown tokens.

The forward pass looks like this: token IDs go through an embedding layer that maps each word into a 256-dimensional vector. Because self-attention has no built-in sense of word order, I add a fixed sinusoidal positional encoding on top of the embeddings — sine and cosine waves at different frequencies, computed once and never updated during training. From there, the sequence passes through two `nn.TransformerEncoder` layers, each with four attention heads and a feed-forward dimension of 512. After encoding, I run masked mean pooling, averaging every real token's final representation while ignoring padding positions completely. The pooled vector goes into a small classifier: a linear layer down to 64 dimensions, a GELU activation, dropout, then a final linear layer that outputs one raw logit per option.

I trained this with `BCEWithLogitsLoss`, since each option gets scored independently — correct or not — rather than as a five-way classification. AdamW handled optimization, learning rate 1e-3, weight decay 1e-2, with gradient clipping capped at a max norm of 1.0 so a single bad batch couldn't blow up the weights. Training ran for 5 epochs at batch size 32, and I tracked a cosine annealing schedule so the learning rate would taper off toward the end of training instead of dropping off a cliff.

The results were honest but modest. Validation MAP@3 landed at 0.5840, with accuracy around 0.4620 and macro F1 at 0.4410. I'm not going to pretend those numbers are competitive — they aren't. What they do prove is that the architecture actually works: attention weights are learning something, the positional encoding is doing its job, and the whole pipeline runs end to end without leaning on anyone else's pretrained knowledge. Given a 10k-word vocabulary and only 200 training questions, this is roughly the ceiling I'd expect from a transformer with zero outside knowledge baked in.

2.2 Model 2 — Fine-Tuned BERT

Model 2 flips the entire premise. Instead of learning language from scratch, I start with `bert-base-uncased`, a 12-layer, 110-million-parameter encoder that Hugging Face already pretrained on a massive text corpus. I load it through `AutoModelForMultipleChoice`, a class built specifically for this kind of task — feed it five candidate sequences, get back five scores.

Each question becomes five tokenized sequences: `[CLS] prompt + context [SEP] option [SEP]`, one per candidate answer, all padded or truncated to 128 tokens. BERT processes all five through the same shared encoder weights, and the classification head reduces each sequence's final `[CLS]` representation down to a single scalar logit. Stack those five logits together and you get a `(batch, 5)` tensor — one score per option, per question.

Because this is really a five-way classification problem (pick the one correct option out of five), I used `CrossEntropyLoss` instead of the binary loss from Model 1. Fine-tuning ran with AdamW at a learning rate of 2e-5 — much lower than Model 1's, since I'm nudging pretrained weights rather than learning from zero — plus weight decay of 0.01 and a linear

warmup schedule covering the first 10% of training steps. I capped training at 3 epochs, and instead of just keeping the final epoch's weights, I saved whichever checkpoint had the lowest validation loss. With only 200 training questions, BERT can overfit fast, and watching validation loss instead of MAP@3 gave me a steadier signal for when to stop trusting a checkpoint.

The jump in performance over Model 1 is large. Kaggle leaderboard score came out to 0.74397 MAP@3, comfortably past the 0.73 cutoff. Accuracy climbed to 0.5980, macro F1 to 0.5750, and validation loss dropped to 0.4850. That gap between Model 1 and Model 2 isn't really about architecture cleverness — BERT's attention mechanism is mathematically the same operation I implemented by hand in Model 1. The difference is 110 million parameters worth of pretraining on real-world text, versus a couple hundred training examples and randomly initialized weights.

2.3 Model 3 — LightGBM/XGBoost Relative-Rank Ensemble

Model 3 doesn't use a neural network for the final prediction at all. I turned the problem into a feature engineering exercise instead. For every question-option pair, I compute 27 numeric features and feed them into gradient-boosted decision trees.

The feature set splits into four groups. Lexical features cover word-level TF-IDF cosine similarity, character-level TF-IDF cosine similarity (useful for catching spelling variants and technical abbreviations), BM25 retrieval scores, Jaccard word overlap, and the fraction of option words that show up in the question text. Semantic features come from `all-MiniLM-L6-v2` sentence embeddings — 384-dimensional vectors compared with cosine similarity, which catches cases where the right answer is phrased differently than the question but means the same thing. Structural features are just length statistics: option length, prompt length, and their ratios. The fourth group — relative rank features — turned out to matter more than I expected. For every similarity score, I compute how far that option's score sits from the average score across its five siblings, and where it ranks (1st through 5th) within that group. Raw similarity numbers shift a lot depending on how long or technical a given prompt is, but "is this option the strongest of its five siblings" holds steady across questions, and tree models pick up on that comparison far more easily than on raw magnitudes.

I trained LightGBM (500 trees, learning rate 0.05, 63 leaves) and XGBoost (400 trees, learning rate 0.05, max depth 6) using 5-fold GroupKFold cross-validation, grouped by question ID so all five options from one question always land in the same fold. Splitting them randomly would let the model peek at sibling options during validation, which is a leakage problem I wanted to avoid entirely. LightGBM used balanced class weighting; XGBoost used `scale_pos_weight` tuned for the 1-correct-to-4-wrong imbalance in every question group. Final predictions blend both models' out-of-fold probabilities, with the blend weight swept across a grid search that landed on 55% LightGBM, 45% XGBoost.

This model won. Kaggle leaderboard score hit 0.74608 MAP@3, edging past BERT's 0.74397. Accuracy reached 0.6650, macro F1 hit 0.6480, and validation loss dropped to

0.3920 — the best numbers across all three metrics, all three models. What's notable is that it got there without a single gradient descent step on a neural network. The heavy lifting is all in the feature engineering, especially those relative rank transformations.

3. Computational Efficiency & MAP@3 Trade-Offs

Performance numbers only tell half the story. Training time varied wildly across the three models, and that gap says a lot about the trade-off between raw predictive power and how expensive that power is to obtain.

On a Kaggle T4 GPU, Model 1 trained in about 2.5 minutes. Model 2 took roughly 6 minutes. Model 3 finished in around 45 seconds. That last number surprised me the first time I saw it, but it makes sense once you break down what's actually happening under the hood. Model 1 and Model 2 both run iterative gradient descent — forward pass, backward pass, weight update, repeated across every batch, every epoch. Model 3 fits gradient-boosted trees, which build additively but don't require backpropagation through a deep network at all. Most of Model 3's runtime actually goes into feature extraction — computing TF-IDF vectors, running BM25, encoding MiniLM embeddings — rather than the tree fitting itself.

MAP@3 is where this efficiency argument gets interesting. If training cost scaled directly with performance, BERT's extra 4-5 minutes over Model 1 would need to buy a lot of accuracy to be worth it, and it does — MAP@3 nearly jumps from 0.58 to 0.74. But Model 3, which trains in a fraction of BERT's time, actually edges out BERT's MAP@3 score entirely. That's not a knock against BERT's architecture. It says something about this specific dataset: with only 200 training questions, a feature-based approach with strong relative-rank signals generalizes at least as well as fine-tuning 110 million parameters, and it does it without needing a GPU warm-up or careful learning-rate scheduling.

I also want to be honest about what "training time" hides. The 45-second number for Model 3 doesn't include the time spent loading MiniLM and running it over every question-option pair to generate embeddings, which happens once and gets cached, but does take real wall-clock time on the first pass. Similarly, Model 2's 6 minutes doesn't include the one-time cost of downloading and caching `bert-base-uncased` locally, which I did specifically to avoid runtime download failures mid-training. Comparing raw fit-time numbers is useful, but it's not the complete cost picture for either model.

Step-level metrics tell a slightly different story than the final MAP@3 numbers. Model 2's validation loss kept dropping steadily epoch over epoch, and MAP@3 tracked that drop closely, which is why I trusted a validation-loss checkpoint strategy instead of chasing MAP@3 directly — MAP@3 can jump around from fold to fold in ways that loss curves smooth out. Model 3's out-of-fold MAP@3, tracked across the alpha sweep for blending LightGBM and XGBoost, showed a fairly flat curve around the 0.50-0.60 blend range, meaning the exact blend weight mattered less than just having both models contribute at all. A single-model version (LightGBM alone, or XGBoost alone) scored measurably lower than any blend in that middle range.

4. Hardware Resource Footprint

The figures in this section are architecture-based estimates derived from known parameter counts and typical inference behavior for each model type. They are not profiled measurements taken from an actual monitoring run (no `psutil`, `nvidia-smi`, or memory profiler logs were captured for this project). If a real hardware breakdown is required for your viva or report submission, I'd recommend actually running a profiling pass on your own machine and replacing the numbers below — this section should not be presented as measured data unless you've verified it yourself.

Model 1 is the lightest of the three by a wide margin. With a 10,000-word vocabulary, a 256-dimensional embedding table, and just two transformer encoder layers, the entire parameter count sits in the low millions — small enough to run comfortably on CPU for both training and inference, which matches the project's local-safety requirement of forcing CPU execution outside of Kaggle. Memory footprint at inference time should stay under a few hundred megabytes, dominated mostly by the PyTorch runtime itself rather than the model weights.

Model 2 is the heaviest. `bert-base-uncased` carries 110 million parameters, which works out to roughly 420-440 MB just for the weights in 32-bit precision, before accounting for activation memory during a forward pass. Running this on CPU is technically possible — and required for local smoke tests under this project's compute rules — but it's noticeably slower than GPU inference, especially once you're scoring five sequences per question. For anything beyond quick sanity checks, this model needs GPU access to be practical, which is exactly why the project rules push full training and inference onto Kaggle's cloud GPUs rather than a local machine.

Model 3 sits in an unusual middle ground. The LightGBM and XGBoost models themselves are small — a few hundred trees each, easily under 50 MB combined once serialized. The real memory cost comes from the MiniLM sentence encoder used for feature extraction, which is itself a compact transformer (roughly 80 MB of weights), plus the TF-IDF vectorizers, which scale with vocabulary size but stay well under 100 MB for a dataset this size. Inference for Model 3 means running MiniLM once per question-option pair and then a fast tree traversal, so CPU inference is realistic here in a way it isn't for full BERT inference at scale.

The practical takeaway: if this needs to run somewhere without a GPU — a laptop, a small server, a free-tier hosting plan — Model 1 and Model 3 are the realistic options. Model 2 needs either a GPU or a tolerance for slow CPU inference, which matters a lot for the deployment discussion below.

5. Deployment & Practical Utility

Nothing in the project source material confirms a working deployment currently exists. What follows is a proposed implementation plan for the optional bonus deployment mentioned in

the project guidelines, not a description of an app that's already built and tested. I'd build and test this before claiming it in a report or demoing it live in a viva.

A trained model sitting inside a Kaggle notebook isn't something anyone outside the course can actually use. The project guidelines call out a Streamlit app, Hugging Face Space, or Flask API as an optional bonus for exactly this reason — turning three separate ranking models into something a person can type a question into and get an answer back from.

The most direct path is a Streamlit app that loads Model 3, since it has the lightest resource footprint and doesn't need a GPU to run at reasonable speed. The app would need three things at load time: the fitted TF-IDF vectorizers (saved with `joblib` or `pickle` after training), the MiniLM sentence encoder (loaded fresh from its cached weights, not re-downloaded on every request), and the blended LightGBM/XGBoost models (also serialized after the GroupKFold training run). All of that gets loaded once, cached with `st.cache_resource`, so it doesn't reload on every user interaction.

The user-facing flow is simple: a text box for the question prompt, five text boxes for options A through E, and a submit button. On submit, the app builds the same 27-feature row I used during training — TF-IDF cosine similarity, BM25 score, MiniLM cosine similarity, length ratios, and the relative-rank transformations computed against the other four options in that single submitted question. That feature row goes through the blended model, and the app displays the three highest-scoring options in ranked order, along with their raw confidence scores so a user can see how close the model's second and third guesses were to the top pick.

For local inference, the workflow stays consistent with the CPU-only rule that applies to local development elsewhere in the project: everything runs without CUDA, since none of Model 3's components strictly need a GPU. A single prediction — feature extraction plus tree inference — should complete in well under a second once the models are loaded into memory, since the expensive one-time cost is loading MiniLM and the tree models at startup, not scoring a single new question afterward.

I'd stay away from deploying Model 2 in this kind of lightweight app unless GPU hosting is available. BERT's per-request latency on CPU, especially scoring five sequences per question, would make the app feel sluggish compared to Model 3, and the memory footprint alone makes free-tier hosting tighter. If deployment quality matters more than squeezing out the last bit of MAP@3 performance, Model 3 is the more practical choice to actually ship, even though Model 2 is a close second on raw accuracy.

One more practical note: the no-external-API rule that applies to training also applies to deployment. Everything in this app — the vectorizers, the sentence encoder, the tree models — runs locally inside the app's own process. No call goes out to an external inference service at request time, which keeps the deployed app consistent with the same constraint that shaped the training pipeline in the first place.